OpenDeepSearch with Tool-Aware Middleware for Reliable Web Search Agents

I. INTRODUCTION

Web agents are becoming increasingly important in contemporary intelligent systems, as they allow for automated interaction with online information. They are commonly used for information retrieval,real time monitoring and decision support. From answering simple user queries to assisting with complex research tasks, web agents help users navigate and make sense of the ever changing web.

In order for web agents to be useful they need to process information from multiple resources while simultaneously carrying out multiple tasks. If the agent is not able to handle these requirements efficiently, the performance of the entire system suffers. Trustworthy web agents need to be able to support informed decision making, access the latest information and retrieve correct results in dynamic environments.

Most current web agents operate in a sequential execution paradigm. Although this paradigm is easy to implement, it also poses a number of limitations. Any mistake occurring in the early stages of the process, such as selection of an inappropriate tool or the acquisition of incomplete information, tends to propagate through the execution process. As soon as such a mistake has occurred, the agent continues to execute its plan without any correction. This becomes a serious problem when an agent is dependent on external tools and large language models. This results in redundant actions, wasted computation, and increased costs due to the heavy usage of paid API's, ultimately leading to reduction in overall efficiency of the system.

To enhance contextual understanding in web agents, Open Deep Search was proposed as a framework that leverages open source tools and advanced reasoning strategies. Although this strategy improves and agent's reasoning capabilities over information, it is still dependent on a structured, step by step execution model. Consequently, real-time monitoring and control of behaviour are still not feasible during execution. The current safety measures are rule-driven and pre-defined, making them inadequate for dealing with unforeseen events. Essentially the agent is like a program that can go astray, with corrective measures that are limited by the initial task definition rather than the dynamic execution context.

To address the above challenges, we proposed OpenDeepSearch with Tool-Aware Middleware (ODS-M). This strategy involves the incorporation of an additional controller layer that continuously tracks the execution of the agent in real time. ODS-M continuously monitors how tools are used, the current status of reasoning, and changes in the context

for as they happen to allow for preemptive corrections of the smaller problems turning into larger scale failures. After an error occurs is too late to take corrective actions; therefore, the middleware is there to help the agent throughout its entire execution.

The ability for control to switch from a static prompt designed to a programmable execution environment with ODS-M greatly improves an agents ability to maintain transparency and allows for greater control measures.The design allows for better insight into agent behaviour and enables developers to control workflows more easily. Consequently agents become easier to debug, work with, and optimize for different tasks and settings.

There are four main parts of the ODS-M framework. The first is the Tool Hierarchy Middleware which manages the way tools are used, helping minimize unnecessary API calls to an outside service. The second part is the Dynamic Prompting Middleware that updates how each agent is instructed based on the environment of the current execution context. The third component of the framework is the Context Augmentation Module, which identifies relevant items using both term importance and semantic similarity. Last but not least, the Centralized Context Store keeps track of each execution event, helping create a consistent series of events between stages and making it easier to utilize context that was previously created in later steps. All four of these parts combine to form a comprehensive method for dealing with changing conditions. The reasoning engine that operates the agents is based on open source language models.

Middleware-Based Execution Control

With our middleware-based execution control structure, we can constantly see what the agents are doing when they are running their tasks. This design provides the ability to make changes and intervene as desired, yet continues to allow the agent to perform uninterrupted. The addition of middleware to the execution pipeline allows us to manage the execution of each task with greater granularity and with a smoother execution process.

Hierarchical Tool Coordination

The Hierarchical Coordination of Tools uses a structured method of managing tool operation within the agent. The primary objective of this structure is to reduce the number of API requests that may be made while still providing high levels of accuracy and relevance in retrieving data. This will provide an effective way to improve overall efficiency by reducing the amount of overhead associated with tool usage within an operational environment.

Hybrid Context Augmentation

The Hybrid Context Augmentation methodology consists of two different types of methods for finding candidates. The first stage involves filtering using TF-IDF scores; then, the second stage uses semantic similarity to determine whether the candidates are relevant or not. When these two retrieval methods are combined, they allow for efficient retrieval of high-quality results, which increases the likelihood that an agent will find valuable contextual information.

Dynamic Context Handling

Because of its centralized context storage method and adaptive prompting, Dynamic Context Handling advances the ability for an agent to grow its personality in an impeccable, instantaneous manner -- as the agent is executing specific tasks over time, the agent's behaviors can change at any point based upon the current context rather than relying on static prompt definitions. This will ultimately provide agents with a higher degree of flexibility, responsiveness, and adaptability to a dynamic environment.

II. RELATED WORK

Open Deep Search (ODS) represents a notable advancement in reasoning-based search systems, enabling web searches that provide real-time, up-to-date information in response to user queries [4]. The system follows a multi-stage pipeline: it begins with query rewriting to improve comprehension, orchestrates external web search tools for information gathering, performs URL scraping to augment context, applies re-ranking to filter irrelevant content, and finally passes the refined context to a reasoning agent for response generation. While ODS achieves competitive performance against systems such as Perplexity Sonar Reasoning Pro and GPT-4 Search Preview [9], it relies exclusively on sequential execution and offers limited system-level control and observability. This restriction constrains adaptive behavior modification and error prevention during execution.

Several research directions inform our approach to these limitations. In the domain of agentic reasoning and web search systems, frameworks like ManuSearch [2] and WebThinker have explored transparent, multi-agent architectures to democratize deep search capabilities. These systems emphasize modularity and interpretability but often retain sequential decision-making patterns. WebCoT extends chain-of-thought reasoning to web agents through reflection, branching, and rollback mechanisms, providing more flexible reasoning pathways while still lacking systematic execution control. DeepDive integrates knowledge graphs and multi-turn reinforcement learning to enhance agent reasoning, demonstrating the value of structured knowledge representation in complex information retrieval tasks.

In the area of dynamic adaptation and prompting, recent work has moved beyond static prompt engineering. Dynamic prompting techniques for task-oriented dialogue and chain-of-thought computation adjust instructions based on context. At the same time, research on efficient knowledge graph extraction demonstrates the scalability benefits of adaptive query formulation. These approaches, however, primarily modify inputs rather than implementing runtime system-level control. Middleware solutions for distributed AI systems apply control and observability principles from distributed computing to AI agent architectures [17], offering insights into real-time oversight but not specifically for web search agents.

The emerging field of agentic retrieval-augmented generation (RAG) provides additional architectural inspiration. Surveys of agentic RAG systems and RAG-reasoning architectures

document the integration of autonomous tool use with retrieval mechanisms, while approaches such as RAG-Fusion explore multi-query generation and reciprocal rank fusion to improve retrieval diversity. These systems improve access to information but typically treat retrieval as a component rather than a centrally managed process with hierarchical tool oversight.

Research in context engineering and middleware further supports our design. Agentic Context Engineering treats contexts as evolving playbooks that accumulate and refine strategies through generation, reflection, and curation. File-system abstractions for context engineering propose a persistent infrastructure for managing heterogeneous context artifacts. While these frameworks address context management, they do not fully tackle tool orchestration, error propagation, and real-time observability in web search agents.

Our work combines insights from these areas while addressing a distinct gap: the lack of **system-level middleware control for web search agents**. Unlike sequential frameworks like ODS [4] or prompting-focused adaptations, ODS-M introduces a dedicated middleware layer that provides real-time observability, hierarchical tool management, and dynamic behavior modification. This design extends beyond existing context engineering frameworks by targeting the orchestration and reliability challenges unique to web search agents, delivering both the transparency of multi-agent architectures [2] and the control mechanisms inspired by distributed middleware [17].

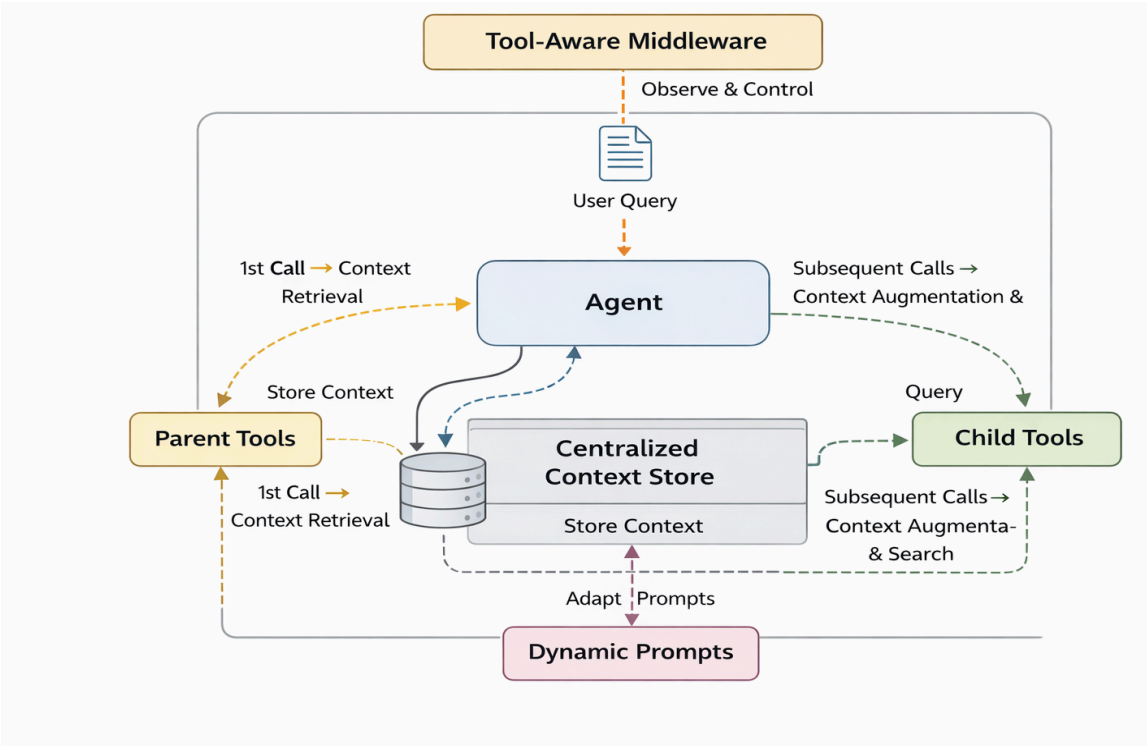
III. METHODOLOGY: OPENDEEPSEARCH WITH TOOL-AWARE MIDDLEWARE

OpenDeepSearch with Tool-Aware Middleware (ODS-M) is an open-source reasoning agentic system designed for web-based information retrieval (IR). The system extends the OpenDeepSearch agent by introducing a middleware layer that enables system-level monitoring and control of the agent's execution. ODS-M integrates an open-source language model to process user queries and interact with external web tools to retrieve the required information. In our experiments, we used the GPT-OSS-120B model as the underlying reasoning model. Given a user query, the agent performs multi-step reasoning over the retrieved web content and generates a final response, whereas the middleware observes and regulates tool usage throughout the execution process.

The ODS-M system comprises four components. These are the Tool Hierarchy Middleware, Dynamic Prompting Middleware, Context Augmentation, and Centralized Context Store. All of these parts work with the help of a special middleware layer that is aware of the tools. ODS-M has the following four components:

A. **Tool Hierarchy Middleware**
B. **Dynamic Prompting Middleware**
C. **Context Augmentation**
D. **Centralized Context Store**

The ODS-M system uses these components to perform its functions. They are coordinated through a tool-aware middleware layer that assists the ODS-M system.



A. **Tool Hierarchy Middleware**

The ODS-M system uses the following structure. It has tools that are like parents and tools that are like children. ODS-M has parent and child tools that work together in this structure.

Parent tools are used to obtain information from the web. We do not use parent tools all the time because they can be expensive and slow to use. Parent tools are used when we first ask for something or after we have used them several times. This helps us reduce costs and obtain information faster. Parent tools are important for obtaining quality information from the web.

Child tools are important. They are like helpers within the system. These child tools do not look outside for information. Instead, they look at the information that is already stored inside the system. In this way, child tools can perform the steps in the thinking process. This helps the agent to use the information it already has, so it does not have to look for it. Child tools are good at handling the steps, which makes the agent work more efficiently. The agent can reuse the information that the child tools have already found.

The system switches between the parent and child tools based on their usage. When the child tool is used much, the parent tool gets reset, and the process starts all over again. Thus, the system can perform many tasks in a row without wasting time or money. It can control what happens. The system also tries to minimize making calls outside of itself, which is good for the parent and child tools.

B. **Dynamic Prompting Middleware**

The ODS-M system has middleware that helps the agent change the system prompt based on what is happening. ODS-M updates its system prompt as things change.

When the agent calls for time, the middleware uses a basic prompt that the system provides to begin figuring things out. The middleware starts with this default system to initiate the reasoning process on the first agent call.

The system is constantly updated. After we use a tool, the middleware removes the data from the tool and updates the system prompt. In this way, the agent always knows what we found out before. The agent stays up to date with updates, so dynamic updating is helpful.

The main goal is to continue improving the system. This helps the agent understand what is going on, make decisions, and answer questions that have many parts more easily. The agent does this by refining the system. In this way, the agent can stay focused on the conversation, figure out what is really being asked, and provide answers to the users questions, especially those that require several steps to answer. The system continues to improve. The agent can understand the context of the conversation and make appropriate decisions. This is very important for the agent to be able to answer questions, such as the system users questions about things.

C. **Context Augmentation**

The third part of the ODS-M is context augmentation. This is where ODS-M uses the results from tools to find URLs. These URLs were then closely examined using web scraping to obtain more information. ODS-M obtains information from each URL. Then, it cleans up and gets rid of anything that is not needed. In this way, the ODS-M has information that is relevant to what it is doing, and it can use this information to make good decisions later on. The ODS-M uses context augmentation to ensure that it has the required information.

1. **Document Chunking**

When we obtain a document, say the document is called ddd, we break it down into parts of text. These parts of the text overlap with each other. We do this so that document ddd makes sense to us.

The document ddd is broken down into chunks, such as C1, C2, C3, and so on, until we have a bunch of these smaller parts. We call these parts chunks of the document ddd. The chunks of document ddd are like C1, C2, C3, and all the way to Cn. These chunks, such as c1 and c2, have some information about their origin. They also have details attached to them when they are made. This is true for all the chunks, not c1 and c2, but all the others too.

2. **Stage 1: TF-IDF-Based Pruning**

To make searching easier, we used a filter that looked at the words in our search. This filter is similar to a tool that helps us determine the importance of each word in our search. We call this tool TF-IDF.

We consider all pieces of text. These were turned into special lists that showed the words in each piece of text. These lists are like maps that help us see what is important in the data. We ignore words that are used a lot, such as "the" and ". They do not help us with our search.

Then, we take what we are searching for, the query. Please turn it into a list. Thus, TF-IDF can help us find what we are looking for by comparing our search query to all the lists of words. Therefore, we used this search map. We compared it with each text piece map. We want to see how similar the search map is to each text piece map. To do this, we calculated the cosine similarity. We calculated the cosine similarity between the search map and each text piece map to determine their similarity. This helps us understand how similar the search map is to each text piece map.

We look at chunks of information. We get rid of the chunks that're not very similar. These chunks have similarity scores that are below the limit that we decide on. The chunks that are left are then sorted by how similar they're to each other. We can also choose to keep the chunks of information that are most similar, to each other. This step helps us filter out the chunks of information quickly based on keywords and reduces the number of chunks of information that we need to look at in this step. The keyword filtering stage is really important. It helps us with the keyword filtering. This stage reduces the number of candidates that we have to look at. We use the keyword filtering to make the process easier. The keyword filtering is what makes this process work.

3. **Stage 2: Semantic Filtering**

The pieces that are left from Stage 1 are processed again. This time we use a filter to see how similar the meanings of the pieces are to each other. We take the query and the candidate pieces from Stage 1. We turn the query and the candidate pieces into vectors. The query and the candidate pieces are like words. We turn these words into vectors. We use a model to do this the model is like a tool that helps us and it is called nomic-embed-text from Ollama. These vectors are like maps that show us where everything is they are, like pictures that help us understand the query and the candidate pieces. We make sure all of these vectors are the length so they can be compared to each other and we can see which query and candidate pieces are similar. We need to see how similar these things are. To do this we use something called cosine similarity. When the query and the candidate pieces have meanings they will be grouped together. We use cosine similarity to find these pieces. The query and the candidate pieces that have the meaning will be together because of cosine similarity.

We look at parts of things that're not similar enough to each other and we get rid of them. The parts of things that are left over are then ranked based on how similar they're in what they

mean. We put the parts of things in order from the ones that mean the same thing, to the ones that mean completely different things and we only keep the best ones. This step really helps us find the things that are related to what we're looking for in terms of meaning, rather than just looking for the same words. We are looking for the things that have the same meaning as the things we are looking for. We do this so we can find similarity in the chunks. The thing that really matters here is the similarity in these chunks. We are looking for similarity in the chunks. This is what is important. The semantic similarity of the chunks is what we care about.

4. **Hybrid Filtering Pipeline**

The filtering pipeline does two things to figure out what is relevant to the query. It takes the query and a bunch of documents. Then it breaks these documents into pieces. After that, it uses the TF-IDF pruner to get rid of the pieces of the documents that do not matter to the query.

The TF-IDF pruner is really good, at getting rid of chunks that we do not need. It does this in an efficient way, which means the TF-IDF pruner helps to eliminate chunks efficiently.

The semantic filter looks at what's left and sorts these pieces in order. It does this based on how they match what we are looking for. This way we get the results and the results that make the most sense. The semantic filter makes sure we see the meaningful results, from our search.

This way of doing things is good because it is fast and it gives us results. We need results when we are dealing with a lot of documents in systems that use retrieval-augmented and middleware-driven methods. The retrieval-augmented methods and middleware-driven methods are what we use to deal with documents. This way of doing things is good, for that.

The filtering pipeline is really good at document filtering because it balances speed and accuracy.

D. **Centralized Context Store**

The fourth part of ODS-M is like a storage room, for context. We use this storage room to keep all the information that we find after we sort things out with the filtering pipeline. When we save the information that we have already looked at on our system the child tools of ODS-M can easily find what they need. This is because the child tools of ODS-M know what the user wants. The child tools of ODS-M can do this quickly because they can look at the information that we have stored in the storage room of ODS-M. This means the ODS-M agent can think about the context we have stored, get the information it needs from the beginning and not have to use paid or closed-source services every time the user asks for something. In essence, the centralized store improves efficiency, lowers costs, and ensures faster, context-aware decision-making.

REFERENCES

[1] S. Alzubi, C. Brooks, P. Chiniya, E. Contente, C. von Gerlach, L. Irwin, Y. Jiang, A. Kaz, W. Nguyen, S. Oh, H. Tyagi, and P. Viswanath, “Open Deep Search: Democratizing Search with Open-Source Reasoning Agents,” *arXiv preprint*, 2025.

[2] L. Huang, Y. Liu, J. Jiang, R. Zhang, J. Yan, J. Li, and W. X. Zhao, “ManuSearch: Democratizing Deep Search in Large Language Models with a Transparent and Open Multi-Agent Framework,” *arXiv preprint*, 2025.

[3] M. Shen and Q. Yang, “From Mind to Machine: The Rise of Manus AI as a Fully Autonomous Digital Agent,” *arXiv preprint*, 2025.

[4] R. Lu, Z. Hou, Z. Wang, H. Zhang, X. Liu, Y. Li, S. Feng, J. Tang, and Y. Dong, “DeepDive: Advancing Deep Search Agents with Knowledge Graphs and Multi-Turn Reinforcement Learning,” *arXiv preprint*, 2025.

[5] J. Nehring, A. Juneja, A. Ahmad, R. Roller, and D. Klakow, “Dynamic Prompting: Large Language Models for Task-Oriented Dialog,” in *Proc. 10th Italian Conf. on Computational Linguistics (CLiC-it 2024)*, Pisa, Italy, Dec. 2024, pp. 644–653.

[6] X. Che, M. Chu, Y. Chen, H. Gu, and Q. Li, “Chain-of-Thought Driven Dynamic Prompting and Computation Method,” *Applied Soft Computing*, vol. 186, Part D, Art. no. 114204, Jan. 2026.

[7] W. Li, Z. Guo, J. Wang, J. Zhang, and G. Zhou, “Dynamic Prompting for Efficient Knowledge Graph Triplet Extraction,” in *Proc. 5th Int. Conf. on Artificial Intelligence, Big Data and Algorithms (CAIBDA 2025)*, Beijing, China, Jun. 20–22, 2025.

[8] A. Singh, A. Ehtesham, S. Kumar, and T. Talaei Khoei, “Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG,” *arXiv preprint arXiv:2501.09136*, 2025.

[9] Y. Li et al., “Towards Agentic RAG with Deep Reasoning: A Survey of RAG-Reasoning Systems in LLMs,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 12120–12145.

[10] Z. Rackauckas, “RAG-Fusion: A New Take on Retrieval-Augmented Generation,” *International Journal on Natural Language Computing (IJNLC)*, vol. 13, no. 1, Feb. 2024.

[11] X. Li, J. Jin, G. Dong, H. Qian, Y. Wu, J.-R. Wen, Y. Zhu, and Z. Dou, “WebThinker: Empowering Large Reasoning Models with Deep Research Capability,” *arXiv preprint arXiv:2504.21776*, Oct. 2025.

[12] M. Hu, T. Fang, J. Zhang, J. Ma, Z. Zhang, J. Zhou, H. Zhang, H. Mi, D. Yu, and I. King, “WebCoT: Enhancing Web Agent Reasoning by Reconstructing Chain-of-Thought in Reflection, Branching, and Rollback,” *arXiv preprint arXiv:2505.20013*, Sep. 2025.

[13] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. V. Le, and T. Luong, “FreshLLMs: Refreshing Large Language Models with Search Engine Augmentation,” in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 13697–13720.

[14] Q. Zhang et al., “Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models,” *arXiv preprint arXiv:2510.04618*, Oct. 2025.

[15] S. R. Rayarao and N. Donikena, “Agentic AI Context Engineering: Patterns, Offloading Strategies, and Context Window Management for Autonomous Systems,” *Authorea Preprint*, Sep. 2025.

[16] X. Xu, R. Mao, Q. Bai, X. Gu, Y. Li, and L. Zhu, “Everything Is Context: Agentic File System Abstraction for Context Engineering,” *arXiv preprint arXiv:2512.05470*, Dec. 2025.

[17] S. Das, “Model and Agentic AI-Driven Middleware for Distributed Systems Design and Validation,” in *Proc. 26th Int. Middleware Conf. (MIDDLEWARE ’25)*, 2025, pp. 27–28, doi: 10.1145/3721464.3777430.